CHAPTER 4

ANALYZING THE PITFALLS, IDENTIFYING THE DEPENDENCIES, AND APPLYING NORMALIZATIONS

In this chapter, we apply the database normalization process to the **Streametrics** architecture. The SQL command-line prompt outputs are faithfully represented in the terminal logs below exactly as they appear in the MySQL interface.

4.1 Analyse the Pitfalls in Relations

In the initial unnormalized design, all streaming data (Users, Subscriptions, Movies, and Watch Records) is grouped into one large, unstructured "Universal" table called `ott_unnormalized_data`.

- **Data Redundancy:** User details (Name, Plan, Zip Code) are repeated for every movie they watch.
- **Update Anomalies:** If a Subscription Plan's price changes, we must update thousands of rows containing that plan.
- **Insertion Anomalies:** We cannot add a new Movie to the database until a User actually watches it.
- **Deletion Anomalies:** If a user deletes their account, we might lose information about a movie if they were the only person who watched it.

```
MariaDB [ott_platform_db]> SELECT * FROM ott_unnormalized_data;
+--------+-----------+--------------------+----------+-------+--------+----------+------
---+-------------------+
| UserID | UserName | User_Phone_Nos | Sub_Plan | Price | Zip | City | MovieID | Genres |
+--------+-----------+--------------------+----------+-------+--------+----------+------
---+-------------------+
| 101 | Yash Raj | 98765432, 91234567 | Premium | 15.99 | 603203 | Chennai | M1 | Action,
Thriller |
| 102 | Kunal | 99887766 | Standard | 10.99 | 110001 | Delhi | M2 | Sci-Fi |
| 101 | Yash Raj | 98765432, 91234567 | Premium | 15.99 | 603203 | Chennai | M2 | Sci-Fi
|
+--------+-----------+--------------------+----------+-------+--------+----------+------
---+-------------------+
```

4.2 First Normal Form (1NF)

4.2.1: Identify Dependency

A table is in 1NF if it contains only atomic (indivisible) values. In our unnormalized table, the `User_Phone_Nos` and `Genres` columns contain multi-valued, comma-separated lists.

4.2.2: Apply Normalization to 1NF

To achieve 1NF, we split the multi-valued attributes into separate rows so every column holds exactly one value.

```
MariaDB [ott_platform_db]> SELECT * FROM ott_1nf_data;
+--------+----------+-----------+----------+-------+--------+----------+---------+-----
-----+
| UserID | UserName | Phone_No | Sub_Plan | Price | Zip | City | MovieID | Genre |
+--------+----------+-----------+----------+-------+--------+----------+---------+-----
-----+
| 101 | Yash Raj | 98765432 | Premium | 15.99 | 603203 | Chennai | M1 | Action |
| 101 | Yash Raj | 98765432 | Premium | 15.99 | 603203 | Chennai | M1 | Thriller |
| 101 | Yash Raj | 91234567 | Premium | 15.99 | 603203 | Chennai | M1 | Action |
| 101 | Yash Raj | 91234567 | Premium | 15.99 | 603203 | Chennai | M1 | Thriller |
| 102 | Kunal | 99887766 | Standard | 10.99 | 110001 | Delhi | M2 | Sci-Fi |
| 101 | Yash Raj | 98765432 | Premium | 15.99 | 603203 | Chennai | M2 | Sci-Fi |
+--------+----------+-----------+----------+-------+--------+----------+---------+-----
-----+
```

4.3 Second Normal Form (2NF)

4.3.1: Identify Dependency

A table is in 2NF if it is in 1NF and contains no **Partial Dependencies**.

- The Composite Primary Key for the 1NF table is `{UserID, Phone_No, MovieID, Genre}`.
- However, attributes like `UserName`, `Sub_Plan`, `Zip` depend *only* on a subset of the Key (`UserID`). This is a partial dependency.

4.3.2: Apply Normalization to 2NF

We separate the table into sub-tables where non-key attributes depend on the *entire* Primary Key.

```
MariaDB [ott_platform_db]> SELECT * FROM users_2nf;
+--------+-----------+----------+-------+--------+----------+
| UserID | UserName | Sub_Plan | Price | Zip | City |
+--------+-----------+----------+-------+--------+----------+
| 101 | Yash Raj | Premium | 15.99 | 603203 | Chennai |
| 102 | Kunal | Standard | 10.99 | 110001 | Delhi |
+--------+-----------+----------+-------+--------+----------+

MariaDB [ott_platform_db]> SELECT * FROM watch_history_2nf;
+--------+---------+
| UserID | MovieID |
+--------+---------+
| 101 | M1 |
| 102 | M2 |
| 101 | M2 |
+--------+---------+
```

4.4 Third Normal Form (3NF)

4.4.1: Identify Dependency

A table is in 3NF if it is in 2NF and contains no **Transitive Dependencies**.

- In `users_2nf`, `City` depends on `Zip` (Zip → City).
- Furthermore, `Price` depends on `Sub_Plan` (Sub_Plan → Price). Both are transitive.

4.4.2: Apply Normalization to 3NF

We remove the transitive variables into their own independent lookup tables.

```
MariaDB [ott_platform_db]> SELECT * FROM users_3nf;
+--------+----------+----------+--------+
| UserID | UserName | Sub_Plan | Zip |
+--------+----------+----------+--------+
| 101 | Yash Raj | Premium | 603203 |
| 102 | Kunal | Standard | 110001 |
+--------+----------+----------+--------+

MariaDB [ott_platform_db]> SELECT * FROM locations_3nf;
+--------+----------+
| Zip | City |
+--------+----------+
| 603203 | Chennai |
| 110001 | Delhi |
+--------+----------+

MariaDB [ott_platform_db]> SELECT * FROM plans_3nf;
+----------+-------+
| Sub_Plan | Price |
+----------+-------+
| Premium | 15.99 |
| Standard | 10.99 |
+----------+-------+
```

4.5 Boyce-Codd Normal Form (BCNF)

4.5.1: Identify Dependency

BCNF dictates that for any dependency `X → Y`, `X` must be a superkey. Assume we have an instructor assigned to a workshop: `(StudentID, Course, Instructor)`.

- Dependency: `Instructor → Course` (Each instructor teaches one specific workshop).
- But `Instructor` is not a candidate key for the whole table. This violates BCNF.

4.5.2: Apply Normalization to BCNF

We break the table so `Instructor` becomes the primary key of a new table.

```
MariaDB [ott_platform_db]> SELECT * FROM student_instructor_bcnf;
+-----------+------------+
| StudentID | Instructor |
+-----------+------------+
| 101 | Dr. Nithya |
| 102 | Prof. Rao |
+-----------+------------+

MariaDB [ott_platform_db]> SELECT * FROM instructor_course_bcnf;
+------------+--------------------+
| Instructor | Course_Module |
+------------+--------------------+
| Dr. Nithya | Database Design |
| Prof. Rao | Media Streaming |
+------------+--------------------+
```

4.6 Fourth Normal Form (4NF)

4.6.1: Identify Dependency

A table is in 4NF if it contains no **Multi-Valued Dependencies (MVD)**.

- In our platform, a single `Movie` can have multiple `Genres` AND multiple `Available_Languages`.
- If stored in one table `(MovieID, Genre, Language)`, we get a Cartesian product because Genre and Language are completely independent facts about the movie.

4.6.2: Apply Normalization to 4NF

We separate the independent multi-valued facts into two distinct tables.

```
MariaDB [ott_platform_db]> SELECT * FROM movie_genres_4nf;
+---------+----------+
| MovieID | Genre |
+---------+----------+
| M1 | Action |
| M1 | Thriller |
+---------+----------+

MariaDB [ott_platform_db]> SELECT * FROM movie_languages_4nf;
+---------+----------+
| MovieID | Language |
+---------+----------+
| M1 | English |
| M1 | Tamil |
+---------+----------+
```

4.7 Fifth Normal Form (5NF)

4.7.1: Identify Dependency

5NF addresses **Join Dependencies** to avoid cyclic reconstruction issues.

- Suppose we have a ternary relationship: `(User, Studio, SubscriptionTier)`.
- If the rule exists: "If a User pays for a Tier, and the Studio supports that Tier, and the User watches that Studio, then `(User, Studio, Tier)` is valid," we suffer from cyclic dependency.

4.7.2: Apply Normalization to 5NF

We decompose the 3-way relationship into three separate 2-way tables. Rejoining these 3 tables perfectly reproduces the combination with no anomalies.

```
MariaDB [ott_platform_db]> SELECT * FROM user_studio_5nf;
+--------+------------+
| UserID | Studio |
+--------+------------+
| 101 | Disney |
| 102 | WarnerBros |
+--------+------------+

MariaDB [ott_platform_db]> SELECT * FROM studio_tier_5nf;
+------------+-------+
| Studio | Tier |
+------------+-------+
| Disney | VIP |
| WarnerBros | Basic |
+------------+-------+

MariaDB [ott_platform_db]> SELECT * FROM user_tier_5nf;
+--------+-------+
| UserID | Tier |
+--------+-------+
| 101 | VIP |
| 102 | Basic |
+--------+-------+
```